

SynTwin: Host-aware digital twin framework for quantitative and context-aware biopart characterization

–User Manual–

Jesús Picó^a, Andrés Arboleda-García^a, David R. Penas^b, Julio R. Banga^b, Alejandro Vignoni^a, Yadira Boada^a, Pavel Zach^a

^a*Synthetic Biology and Biosystems Control Lab, Institut ai2, Universitat Politècnica de València, Camí de Vera s/n, València, 46022, Spain*

^b*Computational Biology Lab, MBG-CSIC, Pazo de Salcedo, Carballeira 8, Pontevedra, 36143, Spain*

Abstract

This document describes SynTwin, the host-aware digital twin framework used in the study “**Host-aware Identification of Intrinsic Gene Expression Biopart Parameters using Combinatorial Libraries**”. SynTwin is designed as a general framework for host-aware characterization and predictive modeling of genetic parts beyond the specific case study presented here. The present release provides the processed experimental data, stored estimation outputs, host-model surrogate, parameter-estimation workflows, practical-identifiability analyses, and visualization scripts used to reproduce the computational results reported in the manuscript and its Supplementary Information. Raw Cytation workbooks are not included in the public distribution; processed experimental inputs are distributed as MATLAB .mat files.



* Jesús Picó

Email addresses: jpico@upv.edu.es (Jesús Picó), maarbgar@upvnet.upv.es (Andrés Arboleda-García), david.rodriguez.penas@csic.es (David R. Penas), j.r.banga@csic.es (Julio R. Banga), alvig2@upv.es (Alejandro Vignoni), yaboa@upv.es (Yadira Boada), pazak@upv.es (Pavel Zach)

Contents

S1 General Overview of SynTwin	3
S2 Software Architecture and Directory Structure	5
S2.1 Top-level modules	5
S2.2 Portable path initialization	6
S2.3 Data availability, attribution, and licensing	6
S3 Optimization Algorithms	7
S4 Experimental Data	8
S4.1 Distribution policy	8
S4.2 pARKA21 background data	8
S4.3 Micro experimental tensors	8
S4.4 Complete experimental tensors	8
S4.5 Relationships among library tensors	8
S5 Parameter Estimation and Result Generation	9
S5.1 Standard executable workflow	9
S5.2 Experimental aggregation levels	9
S5.3 Reduced and full RBS models	9
S5.4 Current library-scale objective function	9
S5.5 Background-model estimation	9
S5.6 Hierarchical library design	10
S5.6.1 Lib24	10
S5.6.2 Lib6	10
S5.6.3 Lib5	10
S5.6.4 Lib30	11
S5.7 Result-tensor construction	11
S5.8 Distribution policy	11
S6 Visualization Workflows	12
S6.1 Experimental-data figures	12
S6.2 Main stored-result figures	12
S6.3 Per-workflow diagnostics	12
S7 Practical Identifiability Analysis	13
S8 Recommended Reproduction Sequence	14
Appendix A Structure of the Experimental Data Tensor (micro version)	15
Appendix B Structure of the Experimental Data Tensor (complete version)	17
Appendix C Structure of the Results Tensor Data Type	22

S1. General Overview of SynTwin

SynTwin implements a host-aware, model-based methodology for the quantitative characterization of genetic bioparts embedded within combinatorial libraries of plasmid-based constitutive gene expression constructs (ECs). The approach enables the estimation of biologically meaningful parameters—plasmid copy number, promoter transcription rate, and ribosome binding site (RBS) translational efficiency—expressed in standardized units with direct biophysical interpretation. These units are grounded on calibrated fluorescence (MEFL) and particle measurements, enabling synthesis rates and inferred parameters to be interpreted in molecular terms rather than as relative or arbitrary quantities. Together, these elements define a host-aware, digital-twin-guided workflow that provides a coherent and scalable framework for quantitative biopart characterization and predictive design of synthetic gene circuits. Fig. S1.1 summarizes the full workflow.

Combinatorial expression construct library and experimental characterization. Each expression construct (EC) consists of a promoter–RBS–GFPmut3–terminator expression cassette embedded in a plasmid backbone carrying a defined replication origin (ORI). The combinatorial design ensures that every biopart appears embedded in multiple distinct genetic and physiological contexts (Fig. S1.1A), which is essential for resolving parameter lack of identifiability that arises when constructs are studied in isolation. For each construct, high-resolution measurements of optical density and GFP fluorescence are used to compute time-resolved specific growth rates and synthesis rates (Fig. S1.1B).

Host-aware modeling and digital twin. To capture the coupling between synthetic gene expression and cell physiology, we employ a host-aware modeling framework combining: (i) an endogenous Host Equivalent Model (HEM) describing resource allocation and growth, and (ii) a mechanistic synthesis model for expression constructs parameterized by biopart-specific quantities.

To improve predictive capacity and mitigate uncertainty inherent to purely mechanistic descriptions, we introduce a hybrid *E. coli* digital twin (Fig. S1.1C) that integrates real-time experimental growth-rate measurements. The specific growth rate, although not providing a complete descriptor of the cellular state, is used as an experimentally accessible physiological observable that constrains the host–circuit interaction in a mechanistically consistent manner Scott et al. (2010); Ceroni et al. (2015). This model-in-the-loop strategy enables the digital twin to infer internal resource fluxes and deliver context-aware predictions of synthesis rates.

Stable parameterization of bioparts across diverse genetic and physiological contexts. The synthesis model explicitly separates: (a) global physiological variables (growth rate, resource flux, substrate availability), determined by the digital twin, from (b) intrinsic biopart parameters (copy number, promoter strength, RBS nominal strength and sensitivity).

While the host state modulates translation capacity, the biopart-specific parameters remain robust with respect to experimental context. This separation enables a quantitative mapping from DNA sequences to biophysical parameters and from these to circuit-level phenotypes.

Parameter estimation via global optimization. Predicted synthesis rates from the digital twin are matched to experimental data through a global optimization procedure (Fig. S1.1D). This allows simultaneous estimation of all biopart parameters using the full combinatorial dataset and propagates uncertainty naturally through Monte Carlo sampling of the solution landscape. The resulting parameter distributions (Fig. S1.1E) are then used to compute confidence intervals for synthesis-rate predictions over the full range of growth conditions.

Role of the combinatorial library. The combinatorial structure of the library is essential for resolving structural and practical identifiability limitations inherent to individual constructs. Because each promoter, RBS, and origin of replication (as the determinant of copy number) appears in multiple distinct contexts, the global sensitivity (Jacobian) matrix becomes structured and effectively full-rank. This enables reliable, robust characterization of all bioparts and supports incremental expansion of the library by adding new parts whose parameters can then be inferred using previously characterized bioparts as anchors. In practice, these anchors are propagated together with their posterior uncertainty, so that incremental characterization is performed relative to an already constrained parameter distribution rather than to a single fixed estimate.

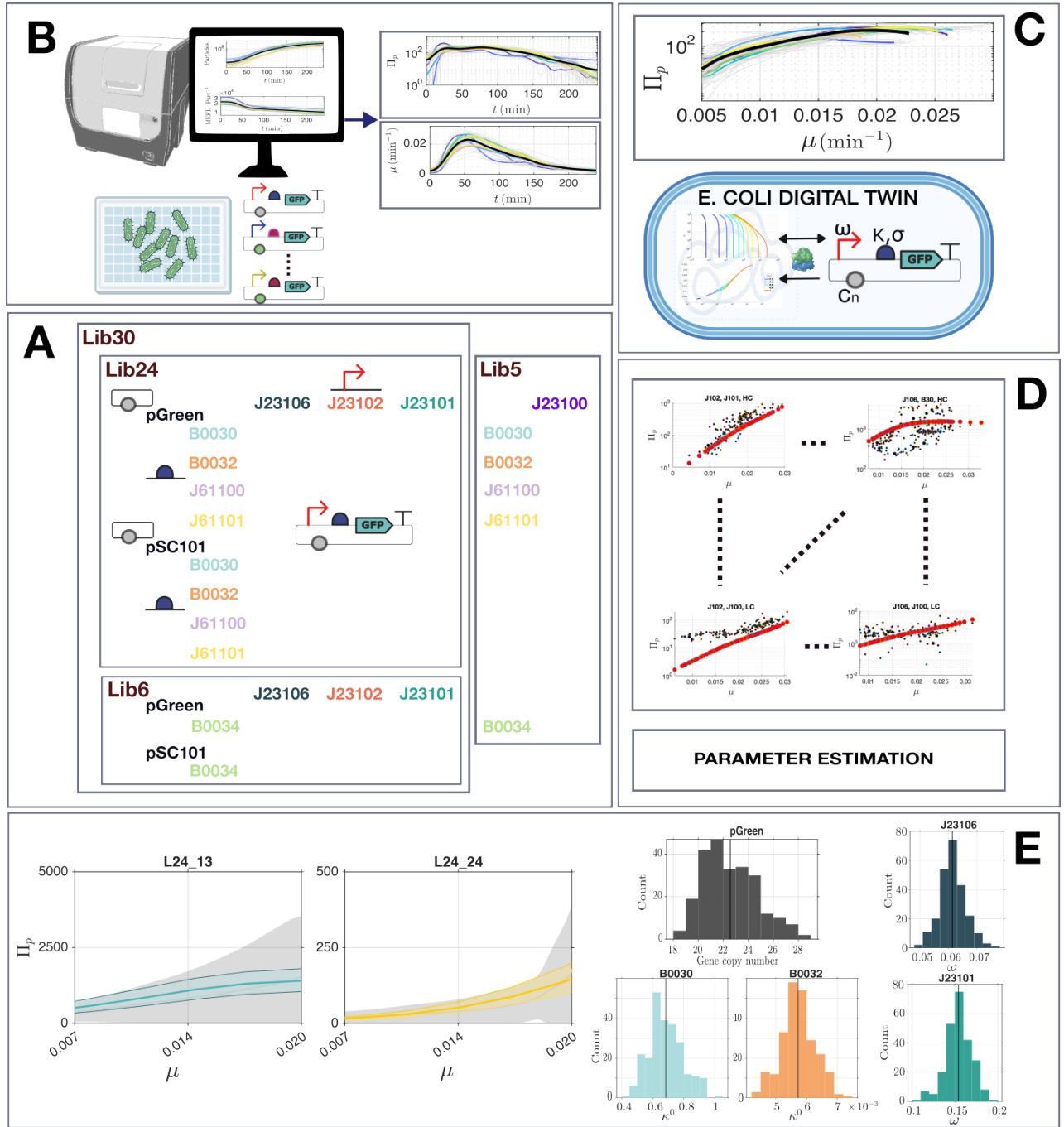


Figure S1.1: Overview of the host-aware biopart characterization workflow. (A) Combinatorial library of plasmid-based constitutive gene expression constructs (ECs) constructed from multiple plasmid backbones carrying distinct origins of replication, promoters, and ribosome binding sites (RBSs), ensuring that each biopart appears across diverse genetic contexts. Plasmid constructs (e.g. pGreen, pSC101) are shown as experimental carriers, while copy number is determined by the underlying origin of replication associated with each plasmid backbone (e.g. pSC101 and ColE1/pMB1-like systems). (B) High-resolution experimental characterization: time-series of optical density and GFP fluorescence are processed to obtain specific growth rate $\mu(t)$ and synthesis rate $\Pi(t)$ for every construct. (C) Host-aware *E. coli* digital twin integrating a mechanistic resource-allocation model with real-time experimental growth-rate measurements. The digital twin provides context-dependent estimates of cellular resource fluxes and predictions of growth-coupled synthesis dynamics. (D) Global parameter estimation combining digital-twin predictions with experimental synthesis-rate data to estimate biologically interpretable biopart parameters (copy number, promoter transcription rate, and RBS translation efficiency). (E) Posterior parameter distributions and synthesis-rate fits across the library. These distributions quantify uncertainty and support a mechanistically grounded characterization of genetic bioparts that remain stable across diverse genetic and physiological contexts.

S2. Software Architecture and Directory Structure

The current SynTwin release is organized as follows:

SynTwin/

Estimation_Pi/

Parameter-estimation, result-generation, and visualization workflows.

Experimental_data/

Processed experimental MATLAB data distributed with SynTwin.

Jacobian_analysis/

Local practical-identifiability and structural incidence analyses.

Scripts_base/

Shared data-processing, model, prediction, and plotting functions.

Show_Data_Exp/

Experimental-data visualization for the 35-construct collection.

Show_Results/

Scripts that reproduce the main stored-result figures without rerunning estimation.

Generate_HEM/

Host Equivalent Model resources, fitting utilities, and surrogate data.

MATLAB/

Third-party MATLAB software used by SynTwin, including BADS and MEIGO/eSS where distributed.

doc/

User and software documentation.

init_SynTwin.m

Portable repository initialization.

SynTwin_path.m

Construction of repository-relative absolute paths.

SynTwin_root.m

Resolution of the SynTwin root directory.

CITATION.cff

Software citation metadata.

LICENSE.md

MIT license for SynTwin code and documentation.

S2.1. Top-level modules

Scripts_base/. Contains the common implementation used across workflows, including the host-aware synthesis prediction functions, Cytation readers, processed-data utilities, reduced- and full-model helpers, and shared plotting routines.

Generate_HEM/. Contains the Host Equivalent Model (HEM), the distributed surrogate `Generate_HEM/HEM_Surrogate/HEM_Surrogate.mat`, host-model fitting and validation scripts, and third-party data used to parameterize the host component.

Experimental_data/. Contains the processed experimental inputs. The public release distributes MATLAB structures rather than raw Cytation workbooks. The current data files are:

- `ExpData_pARKA21.mat`,
- `ExpData_Tensor_lib5_micro.mat`,
- `ExpData_Tensor_lib24_micro.mat`,
- `ExpData_Tensor_lib24.mat`,
- `ExpData_Tensor_lib30_micro.mat`,
- `ExpData_Tensor_lib30.mat`, and
- `ExpData_Tensor_lib35.mat`.

Estimation_Pi/. Contains the background-characterization and biopart-estimation workflows reviewed for this release:

- `pARKA21_background_model/`,
- `L24_ALL_reduced_model/`,
- `L24_ALL_reduced_model_dmA/`,
- `L24_L10_reduced_model/`,
- `L30_ALL_full_model/`,

-
- L30_ALL_reduced_model_dmA/,
 - L6_ALL_reduced_model/,
 - L6_ALL_reduced_model_dmA/,
 - L6_L10_reduced_model/,
 - L5_ALL_reduced_model/, and
 - L5_L10_reduced_model/.

Each subfolder contains a local README specifying its scripts, input tensors, generated variables, parameter inheritance, and distribution policy.

Show_Data_Exp/. Contains `Show_Data_Exp_lib35.m`, which loads `ExpData_Tensor_lib35.mat` and generates the experimental summaries and growth-resolved synthesis-rate plots for the 35 constitutive constructs.

Show_Results/. Contains scripts that regenerate the main result figures from stored data and result tensors. These scripts do not rerun global optimization. They include host-physiology validation, biopart-parameter distributions, and host-aware RBS translation maps.

Jacobian_analysis/. Contains numerical and structural analyses of local practical identifiability, including Jacobian rank, effective rank, condition number, singular vectors, and parameter–construct incidence matrices.

S2.2. Portable path initialization

SynTwin scripts are intended to run without hard-coded absolute paths. Initialize the project once per MATLAB session:

```
ROOT = init_SynTwin();
```

For workflows requiring experimental data and BADS, use:

```
ROOT = init_SynTwin('experimental', true, 'bads', true);
```

A runnable script can add its own directory with:

```
SCRIPT_DIR = fileparts(mfilename('fullpath'));
addpath(SCRIPT_DIR);
```

Repository files should be resolved with `SynTwin_path`, for example:

```
data_file = SynTwin_path( ...
    'Experimental_data', ...
    'ExpData_Tensor_lib30_micro.mat');
```

The capitalization of `Experimental_data` must match the distributed repository on case-sensitive file systems.

S2.3. Data availability, attribution, and licensing

SynTwin distributes the processed experimental data required to inspect and reproduce the reported analyses. Raw Cytation Excel workbooks are excluded from the public MATLAB distribution. The repository-level file `Data Availability & Attribution Statement.md` identifies the third-party data used to fit and validate the HEM, including the Chure and Cremer Flux Parity datasets and upstream values from Bremer and Dennis Bremer and Dennis (2008). SynTwin does not claim ownership of third-party data.

SynTwin interfaces with or redistributes BADS Acerbi and Ma (2017) and MEIGO/eSS Egea et al. (2014). These packages retain their original authorship and licenses. The SynTwin source code and documentation are distributed under the MIT License given in `LICENSE.md`. Citation metadata are provided in `CITATION.cff`.

S3. Optimization Algorithms

The distributed parameter-estimation workflows use Bayesian Adaptive Direct Search (BADs) Acerbi and Ma (2017). BADs is called repeatedly from randomized initial points, normally inside a `parfor` loop. Each run stores the estimated parameter vector, the objective value, and, where retained, the initial point.

MEIGO/eSS Egea et al. (2014) remains available as third-party software and was used in earlier development and comparative analyses. The current public estimation workflows reviewed in `Estimation_Pi/` use BADs multistart.

For exact reruns, set the random seed before estimation:

```
rng(1, 'twister');
```

When Parallel Computing Toolbox is unavailable, `parfor` can be replaced by `for`, with a corresponding increase in execution time.

S4. Experimental Data

S4.1. Distribution policy

The public release provides processed MATLAB data rather than raw plate-reader workbooks. The raw Cytation files used to regenerate selected datasets are maintainer inputs and are not required by the distributed estimation scripts.

S4.2. pARKA21 background data

ExpData_pARKA21.mat contains the variable Data_pARKA21. The data are generated with background correction disabled, consider_BK=0, because they are used to estimate the background itself. Some Cytation layouts identify this construct as PARKA; the preprocessing workflow canonicalizes that alias as pARKA21.

S4.3. Micro experimental tensors

The files ending in _micro.mat contain the minimum processed data required by the parameter-estimation workflows. They are three-dimensional MATLAB cell arrays of size

$$N_{\text{plasmids}} \times N_{\text{promoters}} \times N_{\text{RBSs}}.$$

Each nonempty cell represents one TU and stores metadata together with growth-rate and synthesis-rate trajectories at three aggregation levels:

Global	One global mean trajectory per construct.
Instances	One mean trajectory per independent experiment.
Wells	Individual well trajectories.

The characterization interval extends from the sample at which $\mu_{\max}$ is attained to the sample at which the maximum particle count, $P_{\max}$, is attained. Vector lengths can therefore differ among constructs and experiments.

The distributed micro tensors are:

- ExpData_Tensor_lib5_micro.mat, with expected dimensions $1 \times 1 \times 5$;
- ExpData_Tensor_lib24_micro.mat, with expected dimensions $2 \times 3 \times 4$; and
- ExpData_Tensor_lib30_micro.mat, with expected dimensions $2 \times 3 \times 5$.

The complete micro schema is given in Appendix Appendix A.

S4.4. Complete experimental tensors

The files without the _micro suffix retain the complete processed hierarchy. In addition to TU metadata and μ -II trajectories, they can contain particle counts, MEFL fluorescence, MEFL/Particles, burden estimates, experiment metadata, and well-level values. The complete schema is given in Appendix Appendix B.

ExpData_Tensor_lib35.mat contains the complete processed data for the 35 constitutive constructs used by Show_Data_Exp_lib35.m. The variable Data_Exp_Tensor_lib35 is stored in a $2 \times 4 \times 5$ cell array; empty cells represent combinations not included in the 35-construct collection.

S4.5. Relationships among library tensors

Lib30 spans two plasmids, three promoters, and five RBSs. Lib24 is the Lib30 subset excluding B0034 and uses the Lib30 RBS indices [1, 2, 4, 5]. Lib6 is the B0034 slice and uses Lib30 RBS index 3. Consequently, no separate Lib6 experimental micro tensor is required. Lib5 uses a dedicated tensor because it introduces J23100 in a fixed pGreen context across five RBS variants.

S5. Parameter Estimation and Result Generation

S5.1. Standard executable workflow

Most subfolders under `Estimation_Pi/` follow three stages:

1. `Estimate_*.m`: run BADS multistart and save raw optimization outputs under `Estimated_results/`.
2. `Generate_Results_*.m`: compile parameter statistics, experimental data, sensitivities, deterministic predictions, and Monte Carlo uncertainty into a result tensor or result structure.
3. `Show_Results_*.m`: load the compiled result and generate figures using shared plotting functions.

Objective functions use the prefix `J5w_` for the library-scale parameter-estimation workflows.

S5.2. Experimental aggregation levels

The library objective functions support `Global`, `Instances`, and `Wells`. The canonical public result tensors use `Wells` unless stated otherwise. The selected aggregation must remain consistent among estimation, result generation, and plotting.

S5.3. Reduced and full RBS models

The reduced model estimates the intrinsic initiation capacity

$$\kappa^0 = \frac{k^0}{\sigma^0}$$

and fixes the inverse scaling parameter

$$\rho^0 = \frac{1}{\sigma^0}.$$

The canonical reduced-model workflows use $\rho^0 = 0.02$. The full Lib30 model estimates both κ^0 and ρ^0 . The pSC101 effective copy number is fixed at $N_{\text{pSC101}} = 5$, whereas the pGreen/pSC101 multiplier is estimated when included in the free parameter vector.

S5.4. Current library-scale objective function

For each predicted and experimental synthesis-rate pair, the residual is evaluated in base-10 logarithmic space:

$$r = \log_{10}(\Pi^{\text{pred}}) - \log_{10}(\Pi^{\text{exp}}). \quad (1)$$

The `J5w` objective applies a pseudo-Huber loss with scale δ :

$$\rho_{\delta}(r) = \delta^2 \left(\sqrt{1 + \left(\frac{r}{\delta}\right)^2} - 1 \right), \quad (2)$$

and a smooth synthesis-rate-dependent weight:

$$w(\Pi^{\text{exp}}) = \sqrt{1 + \log_{10}(1 + \Pi^{\text{exp}})}. \quad (3)$$

The pointwise contribution is

$$w(\Pi^{\text{exp}}) \rho_{\delta}(r). \quad (4)$$

The objective is averaged within the selected data aggregation and across the constructs included in the current fit. In L10 workflows, the final normalization uses the number of training constructs.

S5.5. Background-model estimation

The nonexpressing pARKA21 construct is used to estimate the growth-dependent autofluorescence synthesis-rate background. With $x = 100\mu$, the model is

$$\Pi_{\text{bk}}(\mu) = b_0 + A \left(1 - \exp \left[-\frac{x}{k} \right] \right) + q_1 x + q_2 x^2. \quad (5)$$

The parameter order is $[b_0, A, k, q_1, q_2]$. The objective calculates a mean squared residual within each experimental instance and then averages the valid instance-level costs. The estimation and plotting workflows discard the first four and last three points of each retained trajectory.

The public workflow is:

```
Estimate_pARKA21_background_model
Generate_Results_pARKA21_background_model
Show_Results_pARKA21_background_model
```

It produces `Results_pARKA21_background_Wells.mat` and preserves the downstream-compatible parameter vector `Params_global_BKmodel_mean`.

S5.6. Hierarchical library design

The main reduced-model inference sequence is

$$\text{Lib24} \longrightarrow \text{Lib6} \longrightarrow \text{Lib5}.$$

The matching inherited Monte Carlo realization is used consistently within each downstream optimization run.

S5.6.1. Lib24

Lib24 contains $2 \times 3 \times 4 = 24$ constructs. The ALL reduced model estimates eight parameters: three promoter transcription rates (J23106, J23102, J23101), four RBS κ^0 values (B0030, B0032, J61100, J61101), and one pGreen/pSC101 copy-number multiplier.

The canonical workflows are:

```
Estimate_Lib24_ALL_reduced_model
Generate_Results_Lib24_ALL_reduced_model
Show_Results_Lib24_ALL_reduced_model
```

```
Estimate_Lib24_L1O_reduced_model
Generate_Results_Lib24_L1O_reduced_model
Show_Results_Lib24_L1O_reduced_model
```

The L1O workflow performs 24 folds, each using 23 training constructs. Its compiled tensor is the principal upstream source for Lib6.

The canonical ALL subfolder also includes a fixed- ρ^0 robustness scan over $[0.005, 0.01, 0.02, 0.03, 0.05]$. The separate L24_ALL_reduced_model_dmA/ workflow scans $d_{mA} = [0.15, 0.17, 0.20, 0.22, 0.25] \text{ min}^{-1}$, with 0.20 min^{-1} as reference.

S5.6.2. Lib6

Lib6 contains two plasmids, three promoters, and the shared B0034 RBS. Promoter and effective copy-number samples are inherited from the Lib24 L1O tensor. Only B0034 κ^0 is estimated.

The ALL and L1O workflows are:

```
Estimate_Lib6_ALL_reduced_model
Generate_Results_Lib6_ALL_reduced_model
Show_Results_Lib6_ALL_reduced_model
```

```
Estimate_Lib6_L1O_reduced_model
Generate_Results_Lib6_L1O_reduced_model
Show_Results_Lib6_L1O_reduced_model
```

The L1O workflow performs six folds, each using five training constructs. Its compiled tensor supplies B0034 and pGreen inherited samples to Lib5. A separate d_{mA} workflow repeats the ALL estimation at the five fixed degradation rates listed above.

S5.6.3. Lib5

Lib5 contains pGreen, promoter J23100, and five RBS contexts. It estimates only J23100 promoter strength Ω . The four non-B0034 RBS distributions are inherited from Lib24, B0034 from Lib6, and the effective pGreen copy-number samples from Lib6.

The workflows are:

```
Estimate_Lib5_ALL_reduced_model
Generate_Results_Lib5_ALL_reduced_model
Show_Results_Lib5_ALL_reduced_model
```

```
Estimate_Lib5_L1O_reduced_model
Generate_Results_Lib5_L1O_reduced_model
Show_Results_Lib5_L1O_reduced_model
```

The L1O workflow performs five folds, each using four training constructs.

S5.6.4. *Lib30*

Lib30 contains $2 \times 3 \times 5 = 30$ constructs. The full model estimates 14 parameters: three promoter rates, five RBS κ^0 values, five RBS ρ^0 values, and one pGreen/pSC101 multiplier.

The full-model workflow is:

```
Estimate_Lib30_ALL_full_model
Generate_Results_Lib30_ALL_full_model
Show_Results_Lib30_ALL_full_model
```

The separate L30_ALL_reduced_model_dmA/ workflow estimates the nine reduced-model parameters across the five fixed d_{mA} values.

S5.7. *Result-tensor construction*

Compiled tensors contain the experimental trajectories, TU metadata, parameter distributions, sensitivities, deterministic predictions, and Monte Carlo uncertainty propagation. L1O tensors additionally retain fold-specific fields such as Parameters_local_raw, Parameters_local_mean, Parameters_local_std, and J_local_raw. Pooled L1O distributions summarize all folds and are not an independent ALL fit. Appendix Appendix C describes the general schema.

S5.8. *Distribution policy*

The release includes canonical compiled Wells tensors when practical. Large collections of robustness tensors are omitted when they can be regenerated from distributed BADS outputs. The local README of each estimation subfolder states whether Estimated_results/, the compiled tensor, or both are included.

S6. Visualization Workflows

S6.1. *Experimental-data figures*

Run:

```
Show_Data_Exp_lib35
```

to generate the experimental summaries and growth-resolved $\Pi(\mu)$ panels from `ExpData_Tensor_lib35.mat`. The script uses the shared plotting routine `Plot_ExpData_Tensor_v10.m`.

S6.2. *Main stored-result figures*

The `Show_Results/` directory contains:

```
Plot_Host_Physiology_Model_Validation  
Plot_Biopart_Parameter_Distributions  
Plot_RBS_Host_Aware_Translation_Maps
```

These scripts reproduce the host-physiology validation, biopart-parameter distribution, and host-aware RBS translation-map figures from stored data. They do not rerun parameter estimation.

S6.3. *Per-workflow diagnostics*

Each estimation subfolder contains a `Show_Results_*.m` script that loads the corresponding compiled result tensor and calls a shared plotter. Figures are normally exported to a local `Figures/` directory as PNG and SVG files, with selected manuscript workflows also exporting PDF files.

S7. Practical Identifiability Analysis

For a parameter vector θ and synthesis predictions $\Pi(\mu; \theta)$, the local sensitivity matrix is

$$J_{ij} = \frac{\partial \Pi_i}{\partial \theta_j}. \quad (6)$$

The `Jacobian_analysis/` module evaluates exact numerical rank, SVD-based effective rank, singular-value spectra, condition numbers, approximate parameter correlations, and least-identifiable singular-vector directions.

The principal scripts are:

```
Analyze_Libs_Jacobian_ranks  
Analyze_Lib24_reduced_Jacobian  
Plot_Combinatorial_Library_Jacobian_Structure
```

The first compares Lib30, Lib24, and Lib6 under a common data layer. The second provides detailed absolute and relative Jacobian analyses for the Lib24 reduced model. The third plots the binary parameter–construct incidence structure for the 35-construct combination.

A full-column-rank Jacobian indicates local distinguishability at the analyzed parameter point and data layer. Effective rank and conditioning are scale dependent and must be interpreted together with the exact rank, sensitivity normalization, singular vectors, and aggregation level. These analyses establish local practical-identifiability diagnostics, not global identifiability.

S8. Recommended Reproduction Sequence

A complete reconstruction of the main hierarchical results can be run in the following order:

```
% Background model
Estimate_pARKA21_background_model
Generate_Results_pARKA21_background_model
Show_Results_pARKA21_background_model

% Lib24 characterization
Estimate_Lib24_ALL_reduced_model
Generate_Results_Lib24_ALL_reduced_model
Show_Results_Lib24_ALL_reduced_model

Estimate_Lib24_L1O_reduced_model
Generate_Results_Lib24_L1O_reduced_model
Show_Results_Lib24_L1O_reduced_model

% B0034 characterization
Estimate_Lib6_ALL_reduced_model
Generate_Results_Lib6_ALL_reduced_model
Show_Results_Lib6_ALL_reduced_model

Estimate_Lib6_L1O_reduced_model
Generate_Results_Lib6_L1O_reduced_model
Show_Results_Lib6_L1O_reduced_model

% J23100 characterization
Estimate_Lib5_ALL_reduced_model
Generate_Results_Lib5_ALL_reduced_model
Show_Results_Lib5_ALL_reduced_model

Estimate_Lib5_L1O_reduced_model
Generate_Results_Lib5_L1O_reduced_model
Show_Results_Lib5_L1O_reduced_model

% Independent Lib30 full-model analysis
Estimate_Lib30_ALL_full_model
Generate_Results_Lib30_ALL_full_model
Show_Results_Lib30_ALL_full_model
```

The robustness workflows can be run after their corresponding canonical library workflow. Stored canonical tensors can be plotted directly when rerunning optimization is unnecessary.

Appendix A. Structure of the Experimental Data Tensor (micro version)

Generality Across Libraries

The files `ExpData_Tensor_<Library>_micro.mat` contain the minimum required processed experimental data used as input for the host-aware parameter estimation framework.

The structures:

- `ExpData_Tensor_Lib24_micro`
- `ExpData_Tensor_Lib30_micro`
- `ExpData_Tensor_Lib5_micro`

share an identical internal schema. Only the tensor dimensions differ, reflecting the number of TUs in each combinatorial library. This uniform data model enables direct reuse of estimation and digital twin routines across libraries without structural modification.

Tensor Structure

The data structure consists of a three-dimensional MATLAB cell array of size ($\text{num_plasmids} \times \text{num_promoters} \times \text{num_rbss}$). Each tensor entry (cell) corresponds to a single transcriptional unit (TU), defined by a unique combination of origin (plasmid), promoter, and RBS.

Three data aggregation levels are considered:

Global	Averaged aggregated values over all the experimental data
Instances	($\langle \text{NumInstances} \rangle \times 1$ cell array): averaged aggregated values per experiment (<i>Instance</i>).
Wells	($\langle \text{NumWells} \rangle \times 1$ cell array) raw well-level data. By default $\langle \text{NumWells} \rangle = 10$

The data vectors at each aggregation level store the specific growth rate $\mu(t)$ and TU synthesis rate $\Pi(t)$ evaluated between the time at which $\mu_{\max}$ is reached, and the time at which the maximum particle number is attained (see Supplementary Information for more details). Therefore, the vector lengths equal the number of sampled time points within this post- $\mu_{\max}$ regime, which defines the host-aware characterization window, obtained at each data aggregation level.

Global hierarchy

Each tensor cell is a MATLAB struct containing:

TU_Ori	Plasmid/origin identifier (e.g., pGreen).
TU_Promoter	Promoter identifier (e.g., J23106).
TU_RBS	RBS identifier (e.g., B0030).
TU_Bioparts	Comma-separated string encoding the triplet (Ori, Promoter, RBS).
TU_Name	Internal construct label (lab dependent)
TU_color_code	RGB visualization code
Mu_mumax_pmax_global_mean	Time-resolved global mean value of growth rate evaluated from $\mu_{\max}$ to $P_{\max}$ averaged across all Instances.
Mu_mumax_pmax_global_std	Standard deviation associated to <code>Mu_mumax_pmax_global_mean</code>
Pi_mumax_pmax_global_mean	Time-resolved global mean value of synthesis rate evaluated from $\mu_{\max}$ to $P_{\max}$ averaged across all Instances.
Pi_mumax_pmax_global_std	Standard deviation associated to <code>Pi_mumax_pmax_global_mean</code>
Instances	$\langle \text{NumInstances} \rangle \times 1$ cell struct containing all experimental observables
Mu_mumax_pmax_instance_mean	Time-resolved mean value of growth rate evaluated from $\mu_{\max}$ to $P_{\max}$ averaged across the $\langle \text{NumWells} \rangle$ wells of the Instance.

Mu_mumax_pmax_instance_std
 Standard deviation associated to Mu_mumax_pmax_instance_mean

Pi_mumax_pmax_instance_mean
 Time-resolved mean value of synthesis rate evaluated from $\mu_{\max}$ to $P_{\max}$ averaged across the <NumWells> wells of the Instance.

Pi_mumax_pmax_instance_std
 Standard deviation associated to Pi_mumax_pmax_instance_mean

Wells
 <NumWells> × 1 cell struct containing Well experimental observables

Mu_mumax_pmax
 Time-resolved growth rate evaluated from $\mu_{\max}$ to $P_{\max}$ for the Well

Pi_mumax_pmax
 Time-resolved synthesis rate evaluated from $\mu_{\max}$ to $P_{\max}$ for the Well

Appendix B. Structure of the Experimental Data Tensor (complete version)

The files `ExpData_Tensor_<Library>.mat` contain the complete processed experimental data obtained as general input for the host-aware parameter estimation framework. The same internal organization applies to all libraries; only the tensor dimensions change according to the number of transcriptional units (TUs) in each library. The tensor is a 3D cell array: $(\text{num_plasmids} \times \text{num_promoters} \times \text{num_rbss})$. Each cell contains a struct with the following hierarchy:

1. Global hierarchy

- TU_Ori
 - Origin of replication
- TU_Promoter
 - Promoter identifier
- TU_RBS
 - RBS identifier
- TU_Bioparts
 - Concatenated construct string
- TU_Name
 - Internal construct label (lab dependent)
- TU_color_code
 - RGB visualization code
- Data
 - 1×1 struct containing all experimental observables
 - Construct_name
 - Internal construct label (lab dependent, e.g. the same as TU_Name)
 - Global_stats_Mumax_Pi_p
 - 2×2 matrix containing the global averaged maximum specific growth rate $\mu_{\max}$, the synthesis rate $\Pi_{\max}(\mu_{\max})$ and their standard deviations
 - Global_stats_Mumax_Phi_h
 - 2×2 matrix containing the global averaged maximum specific growth rate $\mu_{\max}$, the host cell burden $\Phi_{h\max}(\mu_{\max})\%$ and their standard deviations (see Supplementary Information S3.4)
 - Global_stats_Mumax_Phi_s
 - 2×2 matrix containing the global averaged maximum specific growth rate $\mu_{\max}$, the strain cell burden $\Phi_{s,\max}(\mu_{\max})\%$ and their standard deviations (see Supplementary Information S3.4)
 - Stats_Particles
 - 1×1 struct containing the cell count measurements as *Particles* (see Supplementary Information S2.2) and additional experimental data and metadata (see point 2 below)
 - Stats_MEFL
 - 1×1 struct containing fold-level fluorescence measurements as *MEFL* (see Supplementary Information S2.2 and point 3 below)
 - Stats_MEFL_Particles
 - 1×1 struct containing fold-level derived values of *MEFL/Particles* (see Supplementary Information S2.2 and point 4 below)
 - Stats_Mu
 - 1×1 struct containing fold-level derived values of the specific growth rate μ (see Supplementary Information S2.3) and related singular values and statistics (see point 5 below)
 - Stats_Pi_p
 - 1×1 struct containing fold-level derived values of the background-compensated synthesis rate Π (see Supplementary Information S2.3) and related statistics (see point 6 below)
 - Stats_Phi_s
 - 1×1 struct containing fold-level derived values of the strain cell burden and related statistics (see point 7 below)
 - Stats_Phi_h
 - 1×1 struct containing fold-level derived values of the host cell burden and related statistics (see point 7 below)
 - Global_data_time
 - $n_T \times 1$ vector containing the global time sampling instants, with $n_T = \min_i(n_i)$ obtained as the minimum length of all the experimental instances of the TU.

2. Stats_Particles hierarchy Stats_Particles is a 1×1 struct containing the cell count measurements as *Particles* and additional experimental data and metadata, with fields:

List_instances

$1 \times \text{Num_instances}$ cell containing single experiment (*Instance*) data and metadata. Each cell contains:

Experiment_IDnum

Experiment identification number (lab dependent)

Experiment_date

Experiment date

Colony_number

Colony number (if appropriate)

Status_exp

Experiment status. Set to 0 to indicate experiments not to be included in the evaluations (e.g. because of experimental issues)

Data_time

$n_t \times 1$ vector with the sampling time instants of the experiment

Data_raw

$n_t \times n_{\text{wells}}$ matrix with the *Particles* samples for each well

Data_mean

$n_t \times 1$ vector with the mean *Particles* samples averaged across wells

Data_std

$n_t \times 1$ vector with the standard deviation of *Particles* samples averaged across wells

Data_Mumax_Pi_p

$2 \times n_{\text{wells}}$ matrix containing the maximum specific growth rate μ_{max} for each well (upper row) and the synthesis rates $\Pi_{\text{max}}(\mu_{\text{max}})$ evaluated at μ_{max} (lower row).

Data_Mumax_Phi_s

$2 \times n_{\text{wells}}$ matrix containing the maximum specific growth rate μ_{max} for each well (upper row) and the strain burden $\Phi_{s,\text{max}}(\mu_{\text{max}})$ evaluated at μ_{max} (% , lower row).

Data_Mumax_Phi_h

$2 \times n_{\text{wells}}$ matrix containing the maximum specific growth rate μ_{max} for each well (upper row) and the host burden $\Pi_{h,\text{max}}(\mu_{\text{max}})$ evaluated at μ_{max} (% , lower row).

Data_stats_Mumax_Pi_p

2×2 matrix containing the $\mu_{\text{max}}, \Pi_{\text{max}}(\mu_{\text{max}})$ mean values averaged across the n_{wells} wells (upper row) and their corresponding standard deviations (lower row).

Data_stats_Mumax_Phi_s

2×2 matrix containing the $\mu_{\text{max}}, \Phi_{s,\text{max}}(\mu_{\text{max}})$ mean values averaged across the n_{wells} wells (upper row) and their corresponding standard deviations (lower row).

Data_stats_Mumax_Phi_h

2×2 matrix containing the $\mu_{\text{max}}, \Phi_{h,\text{max}}(\mu_{\text{max}})$ mean values averaged across the n_{wells} wells (upper row) and their corresponding standard deviations (lower row).

Value_Particlesmax_wells

$1 \times n_{\text{wells}}$ vector containing the maximum value of *Particles* P_{max} attained in each well

Index_Time_Particlesmax_wells

$1 \times n_{\text{wells}}$ vector containing the time samples at which the maximum value of *Particles* is attained

Value_Particlesmax_mean

Mean value of P_{max} across the n_{wells} wells.

Index_Time_Particlesmax_mean

Average time sample at which the maximum value of *Particles* P_{max} is attained

Data_mumax_pmax_mean

$n_r \times 1$ vector with the time-resolved value of *Particles*, averaged across the n_{wells} wells of the Instance (experiment), evaluated at the n_r -length samples interval between μ_{max} and P_{max} .

Data_mumax_pmax_std

$n_r \times 1$ vector with the standard deviations for Data_mumax_pmax_mean

Data_Mumax_Pi_p_All

$4 \times (n_{\text{wells}} * \text{Num_instances})$ matrix containing the maximum specific growth rate μ_{max} for each well (first row), synthesis rates $\Pi_{\text{max}}(\mu_{\text{max}})$ (second row), Experiment_IDnum (third row), and Colony_number (fourth row).

Data_Mumax_Phi_s_All

$4 \times (n_{\text{wells}} * \text{Num_instances})$ matrix containing the maximum specific growth rate μ_{max} for each well (first row),

strain burden $\Phi_{s,\max}(\mu_{\max})$ (%), second row), Experiment_IDnum (third row), and Colony_number (fourth row).

Data_Mumax_Phi_h_All
 $4 \times (n_{\text{wells}} * \text{Num_instances})$ matrix containing the maximum specific growth rate $\mu_{\max}$ for each well (first row), host burden $\Phi_{h,\max}(\mu_{\max})$ (%), second row), Experiment_IDnum (third row), and Colony_number (fourth row).

Global_data_mean
Time-resolved mean value of Particles averaged across experiments

Global_data_std
Standard deviation associated to Global_data_mean

Global_value_Particlesmax_mean
Global mean value of $P_{\max}$ averaged across experiments.

Global_index_Time_Particlesmax_mean
Average time sample at which Global_value_Particlesmax_mean is attained

Mumax_pmax_global_mean
Time-resolved global mean value of Particles evaluated from $\mu_{\max}$ to $P_{\max}$ (Data_mumax_pmax_mean) averaged across all Instances (experiments).

Mumax_pmax_global_std
Standard deviation associated to Mumax_pmax_global_mean

3. Stats_MEFL_hierarchy Stats_MEFL is a 1×1 struct containing the fluorescence measurements as *MEFL* and related statistics, with fields:

List_instances
 $1 \times \text{Num_instances}$ cell containing single experiment (*Instance*) data. Each cell contains:

Data_raw
 $n_t \times n_{\text{wells}}$ matrix with the *MEFL* samples for each well

Data_mean
 $n_t \times 1$ vector with the mean *MEFL* samples averaged across wells

Data_std
 $n_t \times 1$ vector with the standard deviation of *MEFL* samples averaged across wells

Data_mumax_pmax_mean
 $n_r \times 1$ vector with the time-resolved value of *MEFL*, averaged across the n_{wells} wells of the Instance (experiment), evaluated at the n_r -length samples interval between $\mu_{\max}$ and $P_{\max}$.

Data_mumax_pmax_std
 $n_r \times 1$ vector with the standard deviations for Data_mumax_pmax_mean

Global_data_mean
Time-resolved mean value of *MEFL* averaged across experiments

Global_data_std
Standard deviation associated to Global_data_mean

Mumax_pmax_global_mean
Time-resolved global mean value of *MEFL* evaluated from $\mu_{\max}$ to $P_{\max}$ (Data_mumax_pmax_mean) averaged across all Instances (experiments).

Mumax_pmax_global_std
Standard deviation associated to Mumax_pmax_global_mean

4. Stats_MEFL_Particles_hierarchy Stats_MEFL_Particles is a 1×1 struct containing the cell specific fluorescence measurements as *MEFL/Particles* and related statistics, with fields:

List_instances
 $1 \times \text{Num_instances}$ cell containing single experiment (*Instance*) data. Each cell contains:

Data_raw
 $n_t \times n_{\text{wells}}$ matrix with the *MEFL/Particles* samples for each well

Data_mean
 $n_t \times 1$ vector with the mean *MEFL/Particles* samples averaged across wells

Data_std
 $n_t \times 1$ vector with the standard deviation of *MEFL/Particles* samples averaged across wells

Global_data_mean
Time-resolved mean value of *MEFL/Particles* averaged across experiments

Global_data_std

Standard deviation associated to Global_data_mean

5. Stats_Mu hierarchy Stats_Mu is a 1×1 struct containing the specific growth rate data (μ) and related statistics, with fields:

List_instances

$1 \times \text{Num_instances}$ cell containing single experiment (*Instance*) data. Each cell contains:

Data_raw

$n_t \times n_{\text{wells}}$ matrix with the growth rate samples for each well

Data_mean

$n_t \times 1$ vector with the mean growth rate samples averaged across wells

Data_std

$n_t \times 1$ vector with the standard deviation of growth rate samples evaluated across wells

Values_Mumax_wells

$1 \times n_{\text{wells}}$ vector containing the maximum value of growth rate μ_{max} attained in each well

Indices_Times_Mumax_wells

$1 \times n_{\text{wells}}$ vector containing the time samples at which μ_{max} is attained

Value_Mumax_mean

Mean value of μ_{max} across the n_{wells} wells.

Index_Time_Mumax_mean

Average time sample at which μ_{max} is attained

Data_mumax_pmax_mean

$n_r \times 1$ vector with the time-resolved value of growth rate μ , averaged across the n_{wells} wells of the Instance (experiment), evaluated at the n_r -length samples interval between μ_{max} and P_{max} .

Data_mumax_pmax_std

$n_r \times 1$ vector with the standard deviations for Data_mumax_pmax_mean

Global_data_mean

Time-resolved mean value of growth rate averaged across experiments

Global_data_std

Standard deviation associated to Global_data_mean

Mumax_pmax_global_mean

Time-resolved global mean value of Data_mumax_pmax_mean (growth rate evaluated from μ_{max} to P_{max}) averaged across all Instances (experiments).

Mumax_pmax_global_std

Standard deviation associated to Mumax_pmax_global_mean

Global_value_Mumax_mean

Global mean value of μ_{max} averaged across experiments.

Global_index_Time_Mumax_mean

Average time sample at which Global_value_Mumax_mean is attained

6. Stats_Pi hierarchy Stats_Pi is a 1×1 struct containing the synthesis rates Π and related statistics, with fields:

List_instances

$1 \times \text{Num_instances}$ cell containing single experiment (*Instance*) data. Each cell contains:

Data_raw

$n_t \times n_{\text{wells}}$ matrix with the time-resolved evaluation of Π for each well

Data_mean

$n_t \times 1$ vector with the mean Π samples averaged across wells

Data_std

$n_t \times 1$ vector with the standard deviation of Π samples averaged across wells

Data_mumax_pmax_mean

$n_r \times 1$ vector with the time-resolved value of Π , averaged across the n_{wells} wells of the Instance (experiment), evaluated at the n_r -length samples interval between μ_{max} and P_{max} .

Data_mumax_pmax_std

$n_r \times 1$ vector with the standard deviations for Data_mumax_pmax_mean

Global_data_mean

Time-resolved mean value of Π averaged across experiments

Global_data_std

Standard deviation associated to Global_data_mean

Mumax_pmax_global_mean

Time-resolved global mean value of Π evaluated from $\mu_{\max}$ to $P_{\max}$ (Data_mumax_pmax_mean) averaged across all Instances (experiments).

Mumax_pmax_global_std

Standard deviation associated to Mumax_pmax_global_mean

7. Stats_Phi_s hierarchy Stats_Phi_s is a 1×1 struct containing the strain burden estimation $\Phi_s(\%)$ and related statistics, with fields:

List_instances

$1 \times \text{Num_instances}$ cell containing single experiment (*Instance*) data. Each cell contains:

Data_raw

$n_t \times n_{\text{wells}}$ matrix with the time-resolved Φ_s samples for each well

Data_mean

$n_t \times 1$ vector with the mean Φ_s samples averaged across wells

Data_std

$n_t \times 1$ vector with the standard deviation of Φ_s samples averaged across wells

Global_data_mean

Time-resolved mean value of Φ_s averaged across experiments

Global_data_std

Standard deviation associated to Global_data_mean

strut

8. Stats_Phi_h hierarchy Stats_Phi_h is a 1×1 struct containing the host burden estimation $\Phi_h(\%)$ and related statistics, with fields:

List_instances

$1 \times \text{Num_instances}$ cell containing single experiment (*Instance*) data. Each cell contains:

Data_raw

$n_t \times n_{\text{wells}}$ matrix with the time-resolved Φ_h samples for each well

Data_mean

$n_t \times 1$ vector with the mean Φ_h samples averaged across wells

Data_std

$n_t \times 1$ vector with the standard deviation of Φ_h samples averaged across wells

Global_data_mean

Time-resolved mean value of Φ_h averaged across experiments

Global_data_std

Standard deviation associated to Global_data_mean

Appendix C. Structure of the Results Tensor Data Type

The data files *Results_Tensor_<Library>_<EstimationScheme>_<ModelVariant>_<DataAggregation>.m* contain the data structure *Results_Tensor_<Library>_<EstimationScheme>_<ModelVariant>* encoding the complete prediction results of the host-aware characterization framework, together with all associated metadata, parameter-estimation summaries, deterministic digital twin simulations, sensitivity analyses, and Monte Carlo (MC) uncertainty propagation outputs.

This naming convention explicitly specifies the experimental library, the parameter-estimation scheme, the model class, and the data aggregation level used during estimation. Notice the data aggregation level *<DataAggregation>* appears in the name of the Matlab *.mat* file, but not in that of the data structure. For this one, the data aggregation level is included as a field (see point 4 below).

The components of the identifier are defined as follows:

Library Identifier. *<Library>* = {Lib24, Lib30, ...}

Specifies the combinatorial library used for parameter estimation. The number denotes the total number of transcriptional units (TUs) included in the library.

Estimation Scheme. *<EstimationScheme>* = {L10, ALL}

L10 Parameters are estimated using cross-validation (Leave-One-Out), where one TU is excluded per fold.

ALL Parameters are estimated using all TUs simultaneously, without cross-validation.

Model Variant. *<ModelVariant>* = {complete, reduced}

complete Full translation model.

reduced Reduced translation model with fixed translational parameter $1/\sigma^0$.

Data Aggregation Level. *<DataAggregation>* = {Global, Instances, Wells}

Indicates the level at which experimental data were aggregated for parameter estimation:

Global Data aggregated across all experiments and wells.

Instances Data aggregated per experimental instance.

Wells Well-resolved experimental data used directly.

Number of Parameters. The number of TUs in the dataset equals the size of the estimated library (*<Library>*). In addition, the maximum number of parameters to be estimated (*<NumPars>*) depends on the number of intrinsically different bioparts (ORIs, promoters and RBSs) in the library. ORIs and promoters contribute with one parameter each one. Each RBS contributes with 2 parameters in case *<ModelVariant>* = *complete*, and 1 parameter in case *<ModelVariant>* = *reduced*. Thus, for instance, Lib30 (2 ORIs, 3 promoters, 5 RBSs) in which the copy number of one of the ORIs is known *a priori*, contains 14 effective parameters for *<ModelVariant>* = *complete*, and 9 in case *<ModelVariant>* = *reduced*.

Tensor Structure

The data structure consists of a three-dimensional MATLAB cell array of size (num_plasmids × num_promoters × num_rbs). Each entry (cell) corresponds to a single transcriptional unit (TU), defined by a specific combination of origin (plasmid), promoter, and RBS.

Each TU cell contains:

1. Transcriptional Unit Metadata

TU_Ori

Plasmid/origin identifier (e.g., pGreen).

TU_Promoter

Promoter identifier (e.g., J23106).

TU_RBS

RBS identifier (e.g., B0030).

TU_Bioparts

Comma-separated string encoding the triplet (Ori,Promoter, RBS).

TU_Name

Internal TU label.

TU_color

RGB visualization code

2. Global experimental data

The following vectors store global statistics (i.e. means and standard deviations across all the wells in all the experimental instances) for the specific growth rate and the synthesis rate (μ and Π). The values span the range from the sample in which the maximum growth rate is attained until that in which the maximum number of Particles is reached.

`Mu_mumax_pmax_global_mean`

`Mu_mumax_pmax_global_std`

`Pi_mumax_pmax_global_mean`

`Pi_mumax_pmax_global_std`

3. Instance- and Wells-Level Experimental Data and Parametric Sensitivities

The field `Instances` encodes experiment-level information for each independent experiment in which a given transcriptional unit (TU) was evaluated. The number of elements in `Instances` therefore equals the number of independent experiments performed for that TU. For example, if a TU was characterized in five separate experiments, `Instances` will contain five elements. Each `Instances{i}` object stores the complete information corresponding to that TU within experiment i . Within each experiment, measurements were performed across `<NumWells>` wells (e.g. `<NumWells>=10`); accordingly, each instance contains a `Wells` field of size `(<NumWells>×1)`.

3.1 Biological Interpretation of Vector Lengths

The lengths of the μ and Π vectors (e.g., `13×1`) correspond to the number of time samples recorded between:

1. The time at which the culture reaches the maximum specific growth rate ($\mu_{\max}$), and
2. The time at which the maximum particle (cell) number is reached.

Thus, each element of these vectors represents a measurement within the post- $\mu_{\max}$ growth regime, which is the regime used for host-aware characterization and sensitivity analysis as described in the Supplementary Information of the associated publication.

3.2 Instance-Level Fields

Each `Instances{i}` struct contains the `(n×1)` vectors:

`Mu_mumax_pmax_instance_mean`

`Mu_mumax_pmax_instance_std`

`Pi_mumax_pmax_instance_mean`

`Pi_mumax_pmax_instance_std`

where n denotes the number of sampled time points between $\mu_{\max}$ and the maximum particle number $P_{\max}$. These quantities represent the mean and standard deviation across the `<NumWells>` wells of that experiment.

Additionally, the following instance-level sensitivity mappings are provided:

`S_Pi_NA_instance_values`

`S_Pi_Omega_instance_values`

`S_Pi_RBS_k0_sigma0_instance_values`

`S_Pi_RBS_inv_sigma0_instance_values`

These `(n×1)` vectors quantify the sensitivities of the experimental synthesis rate Π with respect to the bioparts parameters at each sampled operating point evaluated using the estimated parameters described in point 6. The definitions of these sensitivities follow the formalism described in the Supplementary Information of the host-aware characterization framework.

3.3 Wells-Level Structure

`Instances{i}.Wells` is a `<NumWells>×1` cell array. Each `Wells{j}` element corresponds to one experimental well within experiment i . Each `Wells{j}` struct contains well-resolved quantities:

Mu_mumax_pmax

Pi_mumax_pmax

S_Pi_NA_well_values

S_Pi_Omega_well_values

S_Pi_RBS_k0_sigma0_well_values

S_Pi_RBS_inv_sigma0_well_values

These represent the primary experimental data and associated sensitivity evaluations at single-well resolution. The instance-level mean and standard deviation fields are computed from these well-level values.

4. Parameter estimation data aggregation flag

- Use_mean: Flag indicating how experimental data aggregation is handled (e.g., 'Instances').

5. Local Parameter Estimates under Leave-One-Out Cross-Validation

A Leave-One-Out (L1O) cross-validation procedure was performed across the TUs in the dataset. For each TU:

- One TU is excluded.
- The remaining TUs are used to estimate the $\langle \text{NumPars} \rangle$ parameters of the model.

The estimation procedure is repeated $\langle \text{NumRuns} \rangle$ times (independent optimization runs, usually set to $\langle \text{NumRuns} \rangle = 10$). Thus, for each leave-one-out configuration, $\langle \text{NumRuns} \rangle$ parameter vectors of length $\langle \text{NumPars} \rangle$ are obtained. These results are stored as:

Parameters_local_raw

($\langle \text{NumRuns} \rangle \times \langle \text{NumPars} \rangle$) matrix containing the $\langle \text{NumPars} \rangle$ estimated parameter vectors obtained when the corresponding TU was left out.

Parameters_local_mean

($1 \times \langle \text{NumPars} \rangle$) vector with the mean of the $\langle \text{NumRuns} \rangle$ parameter estimates for that leave-one-out configuration.

Parameters_local_std

($1 \times \langle \text{NumPars} \rangle$) vector with the corresponding standard deviation of the $\langle \text{NumRuns} \rangle$ parameter estimates.

These quantities describe the variability of the global model parameters under the specific L1O configuration associated with this TU cell. Notice under Leave-One-Out Cross-Validation each parameter is estimated a total number of $\langle \text{NumEstimates} \rangle = \langle \text{NumRuns} \rangle \times \langle \text{NumTUs} \rangle$

6. TU-Relevant Biophysical Quantities Aggregated Across All Estimations

Across the estimation procedure $\langle \text{NumEstimates} \rangle$ are obtained per parameter vector, being $\langle \text{NumEstimates} \rangle = \langle \text{NumRuns} \rangle$ in case $\langle \text{EstimationScheme} \rangle = \text{ALL}$, and $\langle \text{NumEstimates} \rangle = \langle \text{NumTUs} \rangle \times \langle \text{NumRuns} \rangle$ in case $\langle \text{EstimationScheme} \rangle = \text{L1O}$. From these $\langle \text{NumEstimates} \rangle$ global parameter vectors, only the parameters that are biophysically relevant for the specific TU corresponding to this cell are extracted and retained. These TU-relevant parameters are mapped into interpretable biophysical quantities, which are stored both in raw sampled form and as aggregated statistics:

Gene_cn_raw

($\langle \text{NumEstimates} \rangle \times 1$)

Gene_cn_mean

Gene_cn_std

Omega_raw

($\langle \text{NumEstimates} \rangle \times 1$)

Omega_mean

Omega_std

RBS_k0_sigma0_raw

($\langle \text{NumEstimates} \rangle \times 1$)

RBS_k0_sigma0_mean

RBS_k0_sigma0_std

RBS_inv_sigma0_raw
($\langle \text{NumEstimates} \rangle \times 1$). Only for $\langle \text{ModelVariant} \rangle = \text{complete}$

RBS_inv_sigma0_mean
Only for $\langle \text{ModelVariant} \rangle = \text{complete}$

RBS_inv_sigma0_std
Only for $\langle \text{ModelVariant} \rangle = \text{complete}$

Notice under Leave-One-Out Cross-Validation, the $\ast_raw$ vectors therefore contain $\langle \text{NumEstimates} \rangle$ samples derived from the full set of L1O estimations ($\langle \text{NumTUs} \rangle$ folds $\times$ $\langle \text{NumRuns} \rangle$ runs).

These values represent the distribution of TU-specific inferred quantities under cross-validated parameter uncertainty. The corresponding mean and standard deviation fields provide compact summary statistics of this distribution. This separation between: (1) $\text{Parameters_local}_\ast$ (fold-specific global model parameters), and (2) TU-specific biophysical quantities aggregated across all folds, ensures that both fold-level estimation variability and global cross-validated uncertainty are explicitly preserved in the data structure.

7. Global Pi-Dependent Mapping Values

These vectors characterize global sensitivity evaluations (see also point 3):

S_Pi_NA_global_values

S_Pi_Omega_global_values

S_Pi_RBS_k0_sigma0_global_values

S_Pi_RBS_inv_sigma0_global_values

8. Deterministic Synthesis Predictions Across Growth-Rate Slices

The field Mu_slices_values (1×10 double) defines a discrete set of growth-rate values (μ) used to interrogate the digital twin of the host-aware model. These values span the post- μ_max regime considered relevant for host-aware characterization and resource-allocation analysis. Given that global model parameters have already been estimated, the calibrated digital twin can be simulated at each prescribed μ slice. This yields deterministic predictions of synthesis rates and internal physiological quantities for the transcriptional unit (TU) represented in the current tensor cell. These predictions are stored in the struct $\text{Synthesis_predictions}$ (1×1 struct).

For each μ slice, the following $\langle \text{NumWells} \rangle \times 1$ vectors are provided:

Mu_values

The discrete growth-rate values at which simulations are evaluated (identical to Mu_slices_values).

Pi_pred_values

Predicted synthesis rate (P_i) at each μ slice.

fs_pred_values

Predicted normalized substrate fraction (f_s), representing resource allocation to growth-related processes.

varphi_pred_values

Predicted effective resource flux (φ), describing intracellular resource allocation dynamics.

load_pred_values

Estimated translational load imposed by the TU at each μ value.

KA_t_pred_values

Effective translation rate (ETR)-related prediction, reflecting translation efficiency under the inferred parameterization.

KRBS_c_values

Effective RBS-related kinetic coefficient derived from the model.

Svarphi_KA_t_values

Composite quantity capturing the interaction between resource flux (φ) and translation rate (K_A^t).

Together, these quantities describe the mechanistic state of the digital twin under varying growth conditions. They provide a deterministic mapping from growth rate (μ) to synthesis rate (Π) and to internal host-aware resource variables.

8.1. Relative Parametric Sensitivities Across Growth-Rate Slices

In addition to deterministic predictions, relative parametric sensitivities are computed at each μ slice and stored in `Relative_sensitivities` (1×1 struct)

For each μ value ($\langle \text{NumWells} \rangle \times 1$ vectors), the following sensitivities are provided:

`S_Pi_NA_values`

Sensitivity of predicted Π with respect to gene copy number.

`S_Pi_Omega_values`

Sensitivity of predicted Π with respect to the promoter ω parameter governing resource/translation coupling.

`S_Pi_RBS_k0_sigma0_values`

Sensitivity of predicted Π with respect to the RBS intrinsic initiation capacity $\kappa^0 = k^0/\sigma^0$.

`S_Pi_RBS_inv_sigma0_values`

Sensitivity of predicted Π with respect to the inverse sensitivity RBS-related parameter.

These sensitivities are computed using the calibrated digital twin and quantify the local dependence of synthesis rate on mechanistic parameters at each growth condition. Importantly, these values provide a growth-rate-resolved mechanistic sensitivity landscape, complementing:

- Instance- and Wells-level sensitivities (experiment-specific),
- Global Π -dependent experimental mappings (condition-aggregated),
- Monte Carlo uncertainty propagation (distribution-level).

9. Monte Carlo (MC) Uncertainty Propagation

To quantify how parameter uncertainty propagates into synthesis-rate predictions and host-aware physiological variables, a Monte Carlo (MC) uncertainty propagation procedure is performed for each transcriptional unit (TU).

9.1 Parameter Sampling

From the parameter distributions associated with the TU (derived from the $\langle \text{NumEstimates} \rangle$ parameter estimated values), $\langle \text{NumRndSamples} \rangle = 1000$ random samples are drawn. These sampled values are stored as:

`Gene_cn_MC_samples`

($\langle \text{NumRndSSamples} \rangle \times 1$ double)

`Omega_MC_samples`

($\langle \text{NumRndSSamples} \rangle \times 1$ double)

`RBS_k0_sigma0_MC_samples`

($\langle \text{NumRndSamples} \rangle \times 1$ double)

`RBS_inv_sigma0_MC_samples`

($\langle \text{NumRndSSamples} \rangle \times 1$ double). Only for $\langle \text{ModelVariant} \rangle = \text{complete}$

Each index ($k = 1, \dots, \langle \text{NumRndSamples} \rangle$) defines one complete sampled parameter realization for the TU.

9.2 Forward Propagation Through the Digital Twin

For each of the $\langle \text{NumRndSamples} \rangle$ sampled parameter sets, the calibrated digital twin is evaluated exactly as described in Section 8:

- The predefined `Mu_slices_values` ($\langle \text{NumMuSlices} \rangle$ growth-rate values) are used.
- For each μ slice, synthesis rate (Π) is predicted.
- Internal host-aware quantities (f_s , φ , load, ETR-related variables, RBS terms) are computed.
- Relative parametric sensitivities are evaluated.

Each realization is stored as one element of the ($1 \times \langle \text{NumRndSamples} \rangle$ struct) `MC_samples`. Every `MC_samples{k}` struct contains:

`Synthesis_predictions`

Deterministic predictions for that sample

`Relative_sensitivities`

Growth-rate-resolved sensitivities

Thus, `MC_samples` stores $\langle \text{NumRndSamples} \rangle$ full digital-twin realizations, each evaluated across $\langle \text{NumMuSlices} \rangle$ growth-rate slices.

9.3 Distribution Construction Across Growth-Rate Slices

After computing all $\langle \text{NumRndSamples} \rangle$ realizations, the results are reorganized by growth-rate slice. For each μ value, the distribution of predicted synthesis rates across the $\langle \text{NumRndSamples} \rangle$ samples is constructed. These distributions are stored in the $1 \times \langle \text{NumMuSlices} \rangle$ struct `MC_mu_slices`. Each slice j contains:

`Mu_slice`

The corresponding growth-rate value.

Pi_pdf

Kernel density estimate (probability distribution) of predicted synthesis rate Π across the <NumRndSamples> MC realizations.

Pi_pred_mean

Mean predicted synthesis rate.

Pi_pred_std

Standard deviation of predicted synthesis rate.

Pi_pred_q50

Median prediction.

Pi_pred_q25, Pi_pred_q75

25th and 75th percentiles.

Pi_pred_q2p5, Pi_pred_q97p5

2.5th and 97.5th percentiles (approximate 95% credible interval).

9.4 Interpretation

The Monte Carlo procedure therefore provides:

1. Distribution-level uncertainty in synthesis-rate predictions at each growth condition.
2. Growth-rate-resolved credible intervals.
3. Full propagation of cross-validated parameter uncertainty through the mechanistic digital twin.

This complements:

- Deterministic predictions (Section 8),
- Local growth-conditioned sensitivities (Section 8.1),
- Fold-level parameter variability (Sections 5–6),

and yields a comprehensive uncertainty-aware characterization of each transcriptional unit within the library.

Supplemental References

- Acerbi, L., and Ma, W. J. (2017). Practical bayesian optimization for model fitting with bayesian adaptive direct search. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), *Advances in Neural Information Processing Systems* (p. 1834–1844). Curran Associates, Inc. volume 30.
- Bremer, H., and Dennis, P. P. (2008). Modulation of chemical composition and other parameters of the cell at different exponential growth rates. *EcoSal Plus*, 3, 10.1128/ecosal.5.2.3.
- Ceroni, F., Algar, R., Stan, G. B., and Ellis, T. (2015). Quantifying cellular capacity identifies gene expression designs with reduced burden. *Nature Methods*, 12, 415–418.
- Egea, J. A., Henriques, D., Cokelaer, T., Villaverde, A. F., MacNamara, A., Danciu, D.-P., Banga, J. R., and Saez-Rodriguez, J. (2014). MEIGO: an open-source software suite based on metaheuristics for global optimization in systems biology and bioinformatics. *BMC Bioinformatics*, 15, 136.
- Scott, M., Gunderson, C. W., Mateescu, E. M., Zhang, Z., and Hwa, T. (2010). Interdependence of cell growth and gene expression: Origins and consequences. *Science*, 330, 1099–1102.